

CoCo 3 Disk Extended Color BASIC

<https://gemini.google.com/app/d816215e6634a16a>

User prompt: Do the BASIC ROMs in a coco3 contain floppy disk drivers and commands, or are they added later? How does that work? What ROM addresses and enables are used?

Response: The built-in system ROMs in a stock Tandy Color Computer 3 do **not** contain floppy disk drivers or disk commands (DIR, LOADM, DSKINI, COPY, etc.).

Those commands and low-level drivers come entirely from **Disk Extended Color BASIC (DECB)**, which resides in an external 8 KB ROM chip located physically inside the floppy disk controller cartridge (or a modern cartridge emulator like the CoCo Flash, Cloud-9, or DriveWire cartridge).

Here is how the expansion, banking, decoding, and initialization work under the hood.

1. Where the ROMs Live in the Memory Map

The CoCo 3 motherboard houses a single 32 KB ROM (often marked Tandy 26-3132 or similar) containing:

- **Color BASIC** (v1.3, 8 KB)
- **Extended Color BASIC** (v2.0, 8 KB)
- **Super Extended Color BASIC** (v1.0, 16 KB)

In standard 64 KB compatible map mode (with GIME task register MMU disabled at reset, or mapped to Page 0):

Address Range	Size	Stock CoCo 3 ROM Content	Cartridge Disk System (When Inserted)
\$8000 - \$9FFF	8 KB	Color BASIC	Color BASIC
\$A000 - \$BFFF	8 KB	Extended Color BASIC	Extended Color BASIC
\$C000 - \$DFFF	8 KB	<i>Super Extended Color BASIC (Lower 8K)</i>	Disk Extended Color BASIC ROM
\$E000 - \$FEFF	~8 KB	<i>Super Extended Color BASIC (Upper 8K)</i>	System Reset Vectors / Super ECB
\$FF00 - \$FFDF	224 B	Hardware I/O (GIME, PIAs, SAM)	Floppy Controller Registers mapped at \$FF40–\$FF48
\$FFE0 - \$FFFF	32 B	Hardware Vectors (6809 Interrupts)	GIME Interrupt / Reset Vectors

When a disk controller is plugged into the 40-pin cartridge port, its 8 KB ROM occupies the window from **\$C000 to \$DFFF**.

2. Bus Signals and Chip Selects: How **\$C000–\$DFFF** is Enabled

The address decoding for cartridges on the CoCo is handled via dedicated select lines on the cartridge expansion connector:

- **Pin 32: *CTS (Cartridge Select):** The GIME chip (or SAM in CoCo 1/2) decodes CPU read accesses to **\$C000–\$DFFF** and asserts *CTS active low.
- **Pin 8: *CART (Cartridge Detect / Interrupt):** Pulled low by an autostart cartridge or used as an FIRQ source. (Standard disk controllers do not tie this permanently low for autostart, as DECB boots via the standard reset hook).
- **Pin 34: *SCS (Spare Chip Select):** Asserted low when accessing the peripheral I/O page at **\$FF40–\$FF5F**. The Western Digital floppy controller chip (FD1773, WD1793, or WD2797 depending on controller model) uses *SCS along with address lines A0–A3 to map its command/status and data registers to **\$FF48–\$FF4B**.

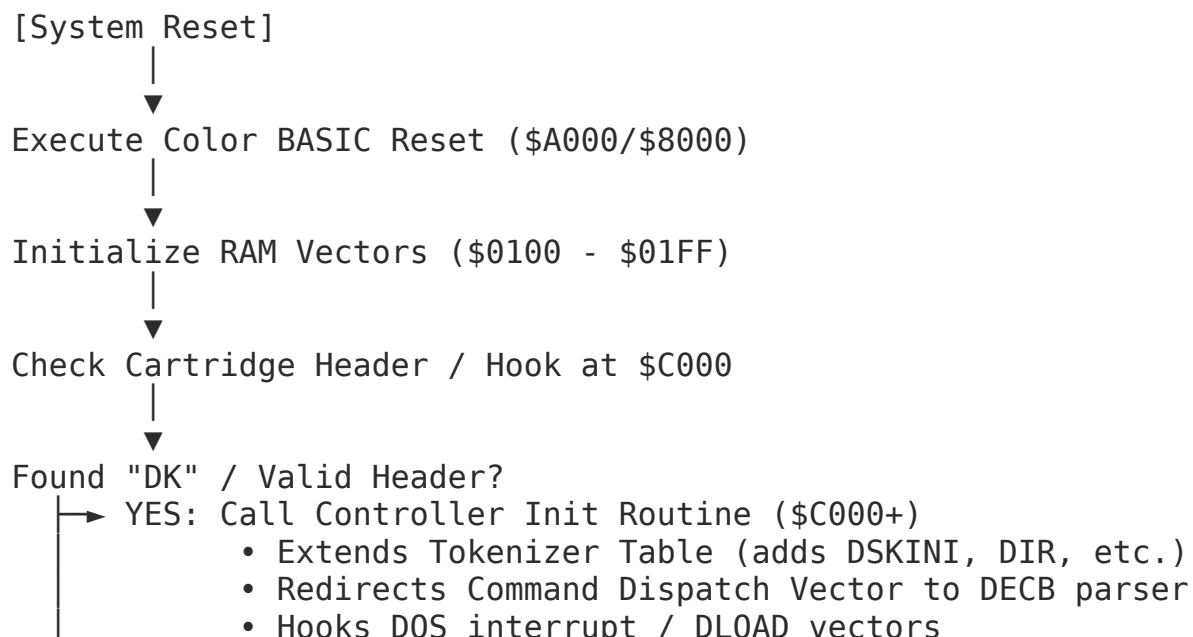
Internal vs. External Conflict:

When the GIME chip generates *CTS for an external cartridge at **\$C000–\$DFFF**, it simultaneously **disables internal motherboard ROM routing** for that 8 KB block.

Super Extended Color BASIC was designed around this: the CoCo 3's initialization routines copy necessary parts of the Super ECB extensions into RAM or swap pages via the GIME MMU before DECB takes over the **\$C000–\$DFFF** window.

3. How DECB Integrates into BASIC at Boot

The integration of the floppy commands does not require patching the stock ROMs directly; instead, it uses built-in **vector hooks** in RAM established during cold boot:



- Initializes WD floppy controller hardware (\$FF40+)

→ NO: Continue normal ROM startup -> OK prompt

1. **Cold Reset:** The 6809 starts at the reset vector (\$FFFE), jumping to internal ROM setup routines.
 2. **Hook Table Initialization:** Basic sets up an execution and parsing jump table in the zero-page / direct-page / low RAM (specifically around \$0150–\$0170).
 3. **ROM Scanning:** The startup code inspects address \$C000. If an external ROM is present, it looks for the characteristic DECB entry point or signature.
 4. **Patching Vectors:** The disk controller's initialization code executes:
 - It splices into the **reserved-word evaluation vector** (command dispatch vector). When the BASIC interpreter encounters a command token it doesn't recognize as standard or Extended Color BASIC, the fallback vector calls the DECB token handler at \$C000+.
 - It updates the system pointers (MEMSIZ) to allocate workspace RAM for disk buffers (Track buffer, File Allocation Tables, Directory buffers).
 - It hooks the DOS command vector.
-

User prompt: what is the Cloud-9 cartridge, and how does it work?

Response: In the TRS-80 / Tandy Color Computer community, "**Cloud-9**" refers to **Cloud-9 Technology** (founded by Mark Marlette), a prominent third-party hardware developer for the CoCo.

Cloud-9 produced two primary cartridge products designed to replace or upgrade vintage floppy controllers: the **miniFLASH** cartridge and the flagship **SuperIDE** interface.

1. The Two Main Cloud-9 Cartridges

- **SuperIDE Interface:** A mass-storage and ROM expansion cartridge. It provides a 40-pin IDE interface, an onboard CompactFlash socket, an optional Real-Time Clock (RTC), and onboard in-system programmable FLASH memory.
- **miniFLASH:** A standalone, non-volatile solid-state ROM cartridge designed to replace traditional UV EPROMs, providing multiple switchable/software-selectable banks for custom DOSes or Program Paks.

2. How the Hardware Works on the CoCo Bus

Both the SuperIDE and miniFLASH rely on standard CoCo cartridge-port signals (*CTS, *SCS, E, and Q) combined with onboard banking logic to sidestep the 6809's 64 KB address limits:

A. The Onboard Flash ROM (Bank Switching)

- **Cartridge Select (*CTS):** When the CPU accesses \$C000–\$DFFF, the CoCo's SAM/GIME chip asserts *CTS active low.
- **Banking Architecture:** Cloud-9 cartridges feature **64 KB of Flash memory split into four 16 KB logical banks** (or eight 8 KB slots).
- **Mapping:** Since the cartridge slot window \$C000–\$DFFF is only 8 KB wide in standard Color BASIC mode, an onboard latch (often a CPLD or discrete logic register) selects high-order address lines on the Flash chip. This allows you to store multiple OS options in non-volatile flash simultaneously:
 - Bank 0: Standard Disk Extended Color BASIC (DECB)
 - Bank 1: **HDB-DOS** (Hard Disk Basic for IDE/CF)
 - Bank 2: NitROS-9 boot kernel
 - Bank 3: Diagnostic utilities or standard game/utility ROM Paks
- **In-System Reprogramming:** Instead of removing an EPROM chip and placing it in an external programmer, the CoCo can run a software utility in BASIC or OS-9 that unmask write pulses to the Flash chip and writes new binary images directly across the cartridge bus.

B. The SuperIDE Storage Interface

The SuperIDE connects standard ATA/IDE drives or CF cards directly to the 8-bit 6809 data bus:

- **Spare Chip Select (*SCS):** The CoCo asserts *SCS when the processor reads or writes to the I/O range \$FF40–\$FF5F.
- **Register Decoding:** The SuperIDE decodes this range to map the IDE drive's task-file registers (Command, Status, Cylinder, Head, Sector Number, and Data) into CPU memory space (typically occupying registers in the \$FF50–\$FF57 or \$FF40–\$FF47 block to avoid colliding with floppy controllers or serial packs).
- **16-bit to 8-bit Bus Bridging:** Standard IDE is 16-bit, while the 6809 bus is 8-bit. The SuperIDE hardware handles the word-to-byte latching automatically, so two 8-bit reads or writes sequence a single 16-bit IDE word transfer.

3. Software Integration (HDB-DOS & NitROS-9)

The hardware is paired with **HDB-DOS** (developed by Allen Huffman and extended for IDE hardware):

1. **Boot Vector Takeover:** On reset, the cartridge ROM at \$C000 is initialized.
2. **Virtual Floppy Drive Emulation:** Because standard Color BASIC only understands floppy tracks and sectors (35 or 40 tracks, 18 sectors/track, 256 bytes/sector), HDB-DOS virtualizes floppies on the IDE/CF card. It treats massive hard drives as an array of thousands of virtual floppy disks:

```
DRIVE 0, 15      ' Mounts Virtual Floppy #15 as Drive 0
DIR              ' Standard DECB DIR command reads CF sector data
```

3. **NitrOS-9 Support:** In OS-9 / NitrOS-9, the ROM kit can hold the base boot kernel (OS9Boot), jumping straight into the OS upon boot without requiring floppy disk media. A dedicated disk driver (rbsuper or ide.dr) addresses the IDE sectors directly for fast file access.

Sources

1. [What's A CoCo - Cloud9Tech](#)
 2. [CoCoWares. All the hardware and software for your Color Computer](#)
 3. [SuperIDE Interface - Cloud9Tech](#)
-

User prompt: How does that compare with the CocoSDC cartridge?

Response: While both the **Cloud-9 SuperIDE** and Darren Atkinson's **CoCoSDC** provide modern mass-storage and flash-based DOS environments, they solve the problem from fundamentally different engineering perspectives:

- **Cloud-9 SuperIDE** is a **hard drive interface** attached directly to the 6809 bus, redirecting disk calls via software patches (HDB-DOS).
- **CoCoSDC** is an **active microcontroller-based floppy drive controller emulator** (recreating the Western Digital WD17xx chip in silicon/firmware) backed by an SD card.

Key Architectural Differences

Feature	Cloud-9 SuperIDE / miniFLASH	Darren Atkinson's CoCoSDC
Core Architecture	CPLD + discrete logic interfacing an ATA/IDE or CF card directly to the 8-bit 6809 bus	High-speed AVR microcontroller (ATmega) reading FAT-formatted SD/SDHC cards
Media Type	40-pin IDE hard drive / CompactFlash card	MicroSD or full-size SD/SDHC card
Hardware Emulation Level	None. Presents IDE task-file registers (LBA/CHS) at \$FF50–\$FF57	Register-level WD17xx controller emulation at \$FF48–\$FF4B (plus control registers at \$FF40)
Compatibility Approach	Software patch level (HDB-DOS intercepts standard DECB vectors)	Dual-mode: Fast software commands <i>plus</i> cycle-accurate hardware register emulation
Copy-Protected & Booter Disks	Fails. Software with custom disk loaders or track-stepping bypasses HDB-DOS and hangs.	Succeeds. Emulates the WD controller so closely that custom fast-loaders and copy-protection work.
Stock DOS	HDB-DOS (Allen Huffman)	SDC-DOS (Darren Atkinson)
Flash ROM Banking	64 KB Flash (4 × 16 KB or 8 × 8 KB banks)	128 KB Flash (8 × 16 KB banks, hardware DIP/software switchable)

Feature	Cloud-9 SuperIDE / miniFLASH	Darren Atkinson's CoCoSDC
Multi-Disk Swapping	Requires typing DOS mount commands	Physical push button on the cartridge to cycle through multi-disk game sets

How CoCoSDC Works Under the Hood

1. Hardware-Level Western Digital Emulation

On a vintage Tandy controller, the 6809 talks to a Western Digital FD1773/WD1793 chip mapped at \$FF48–\$FF4B (via *SCS), issuing low-level commands like *Restore*, *Seek*, *Step*, *Read Sector*, and *Write Sector*.

The CoCoSDC places an ATmega MCU directly on the CoCo's data bus and control lines (*SCS, E, Q, R/W). When the 6809 accesses \$FF48–\$FF4B, the microcontroller intercepts the read/write cycles in real time and emulates the exact internal state machine of the WD17xx controller. If a game uses a custom machine-code sector reader or steps the head manually, the MCU tracks the virtual head position across the mounted .DSK image on the SD card.

2. Dual-Mode Operation (Speed vs. Compatibility)

Because pure track-and-sector bit-banging can introduce latency over SPI to an SD card, **SDC-DOS** uses a hybrid approach:

- **High-Level Hook Mode:** For standard Color BASIC commands (DIR, LOADM, etc.), SDC-DOS bypasses the WD17xx state machine and issues high-speed block read/write commands directly to the MCU, loading files instantly.
- **Low-Level Emulation Mode:** If software takes over the system, disables interrupts, and writes directly to the controller registers at \$FF48, the MCU seamlessly falls back to cycle-accurate WD17xx hardware emulation.

3. Image Mounting and SD Card Usability

With the Cloud-9 SuperIDE, the CF card is formatted into raw HDB-DOS partition geometry; reading or writing individual disk images typically requires special host tools or DriveWire bridges.

With the CoCoSDC:

- The SD card is formatted with standard **FAT16/FAT32**, meaning you can insert it directly into a modern PC or Mac to drag-and-drop .DSK (or .PAK, .ROM) files.
- In BASIC, SDC-DOS provides extended syntax to navigate directories and mount images:

```
DRIVE 0,"GAME.DSK"      ' Mounts the FAT file to virtual Drive 0
DIR                      ' Standard DECB DIR reads the virtual image
```

- SDC-DOS also includes native protocol support for **DriveWire**, allowing it to serve as a bridge without extra serial hardware.

Summary

The **Cloud-9 SuperIDE** is essentially an **IDE/PATA hard disk controller** adapted to a 6809 bus, providing massive linear block storage ideal for extensive NitroS-9 installations or dedicated HDB-DOS partitions.

The **CoCoSDC**, by contrast, is an **intelligent floppy drive simulator**. Because it mimics the WD17xx silicon at the register level, it offers near 100% software compatibility with the historical CoCo floppy catalog—including booter disks, custom loaders, and copy-protected titles—while retaining the convenience of a modern FAT-formatted SD card.

Sources

1. [CoCoSDC - The Tandy/Radio Shack Color Computer Wiki](#)
 2. [CoCoSDC for TRS-80 Color Computer part 1 - Sub-Etha Software](#)
-

User prompt: Does the "Disk Extended Color Basic v1.1" ROM that works on a coco2 also work on a coco3 for providing DISK Basic commands?

Response: **Yes**. Disk Extended Color BASIC (DECB) **v1.1** runs on a CoCo 3 and provides disk commands (DIR, LOADM, DSKINI, etc.). In fact, DECB v1.1 is the standard, most common disk ROM used across the CoCo 1, 2, and 3.

Why It Works Seamlessly

1. **Architecture Neutrality:** DECB was written to operate via the standard Color BASIC / Extended Color BASIC RAM vectors (located in low RAM at \$0100–\$017F). Because the CoCo 3 preserves backward compatibility with those execution vectors, DECB hooks in identically regardless of whether the host machine is a CoCo 1, 2, or 3.
2. **RAM Relocation on the CoCo 3:** Unlike the CoCo 1 and 2—which run DECB directly out of ROM at \$C000–\$DFFF during normal execution—the CoCo 3's startup routines detect the cartridge ROM, allocate RAM blocks, and **copy the ROM image into RAM**. This allows users to patch disk stepping rates or parameters in RAM on a CoCo 3 without modifying the physical chip.
3. **Hardware Registers:** DECB interacts directly with the Western Digital floppy disk controller (FD1773/WD1793/WD2797) mapped at \$FF48–\$FF4B and the drive/latch control registers at \$FF40. The CoCo 3's GIME asserts *SCS for the peripheral space \$FF40–\$FF5F identically to the SAM chip in the CoCo 1/2, ensuring full hardware register compatibility.

Hardware Caveats to Watch For

While the **ROM code itself** is 100% compatible, there are two physical hardware considerations:

- **Tandy Multi-Pak Interface (MPI):** If you use an original Tandy Multi-Pak Interface (26-3024) to connect the disk controller to a CoCo 3, early production revisions of the MPI suffer from clock/bus slew issues (in the PAL/GAL logic) that cause intermittent crashes or lockups on a CoCo 3. A well-known PAL/resistor modification is required for reliable CoCo 3 operation.

- **High-Speed Mode (1.78 MHz):** The standard DECB v1.1 driver routines contain delay loops timed for the standard **0.89 MHz** 6809 bus speed. If you switch the CoCo 3 into 1.78 MHz "turbo" mode (POKE 65497, 0), stock DECB v1.1 disk access can fail or produce I/O errors because the floppy controller's status and data transfer handshake timing gets violated. Modern patched variants (like RGB-DOS or SDC-DOS) automatically drop the CPU to 0.89 MHz during disk I/O and restore 1.78 MHz upon completion.

Sources

1. [CoCo SDC Emulator Overview | PDF | Computer File | Floppy Disk](#)
2. [CoCoSDC | Sub-Etha Software](#)

User prompt: This disk will be emulated, so Hardware Caveats mostly don't apply. What caveats might emulators find, in emulating a Floppy Drive? And if we want a larger Hard Drive mapped in this I/O space, like for NitrOS-9 to use, can Disk Extended Color Basic do anything with them?

Response: Pitfalls in Emulating the CoCo Floppy Controller

Emulating the Western Digital FDC architecture (\$FF40–\$FF4B) goes beyond basic sector-dump lookups. The CoCo's minimal hardware design pushed significant timing and signaling responsibilities directly onto the software and the 6809 bus.

1. The Dynamic \$FF40 Control Latch

Address \$FF40 is not part of the Western Digital silicon; it is an external write-only latch implemented with discrete TTL logic on the controller card. Emulators must track bit mutations carefully:

Bit	Function	Emulation Catch
D0–D1	Drive Select (0–3 decoded)	Some software relies on deselecting drives to reset latch states.
D2	Motor On	Drives won't transfer data or assert index pulses if cleared.
D3	Write Density (MFM vs. FM)	Used by formatters and track loaders to handle single-density gaps.
D4	Precompensation	Usually ignorable in emulation, but reads must not alias registers.
D5	Non-Maskable Interrupt Enable	Critical. Enables routing of the FDC's INTRQ to the 6809's NMI pin.
D6	Head Select	Side 0 vs. Side 1 for double-sided drives.
D7	Halt/DRQ Enable	The hardest part to emulate cleanly.

2. The \$FF40 Bit 7 / DRQ "Halt" Circuit

To stream bytes at 0.89 MHz without a hardware DMA controller, Tandy wired the WD17xx DRQ (Data Request) line to the 6809's hardware HALT input via Bit 7 of \$FF40:

1. The 6809 issues a Read or Write command to the WD controller (\$FF48).
2. The CPU immediately enters a tight read/write loop or writes \$D0 to \$FF40 to enable the Halt circuit.
3. The hardware forces the 6809 to HALT mid-cycle.
4. When the WD controller shifts a complete byte into/out of its data register (\$FF4B), it asserts DRQ.
5. The assertion of DRQ momentarily unhalts the 6809 for just enough clock cycles to execute a single LDB \$FF4B / STB ,X+ sequence before halting it again for the next byte.

Emulator Impact: If an emulator updates the FDC state machine asynchronously or purely on instruction boundaries, software that relies on hardware HALT handshake timing (including DECB formatters and many booter games) will suffer buffer overruns or freeze waiting for the CPU to yield.

3. Low-Level Track Formats vs. Flat .DSK Files

A standard .DSK file is just a flat array of 161,280 bytes (35 tracks × 18 sectors × 256 bytes). However, real software interacts with the WD17xx using raw track commands:

- **Read Track (\$E4) / Write Track (\$F4):** Used by disk copiers and copy-protection routines to read raw sync marks (\$A1), address marks (\$FE, \$FB), and inter-sector gap bytes (\$4E). Flat .DSK containers lack this metadata.
- **Non-Standard Geometries:** Emulators usually need support for .DMK images (David Keil's format), which preserve raw MFM byte streams, ID marks, clock bits, and gap lengths to avoid failing on protected or non-standard media.

4. Motor Spin-Up and Interrupt Latencies

The WD controller reports a BUSY bit in its status register (\$FF48). Real routines poll BUSY after issuing Seek, Step, or Restore commands. If an emulator clears BUSY instantly (zero-cycle seek), software timing loops that calculate track-to-track step delays can miscalculate head settling times and report persistent seek errors.

Can Stock DECB Do Anything with a Large Hard Drive?

In its unpatched state: **No, but with a patched ROM (HDB-DOS), it can.**

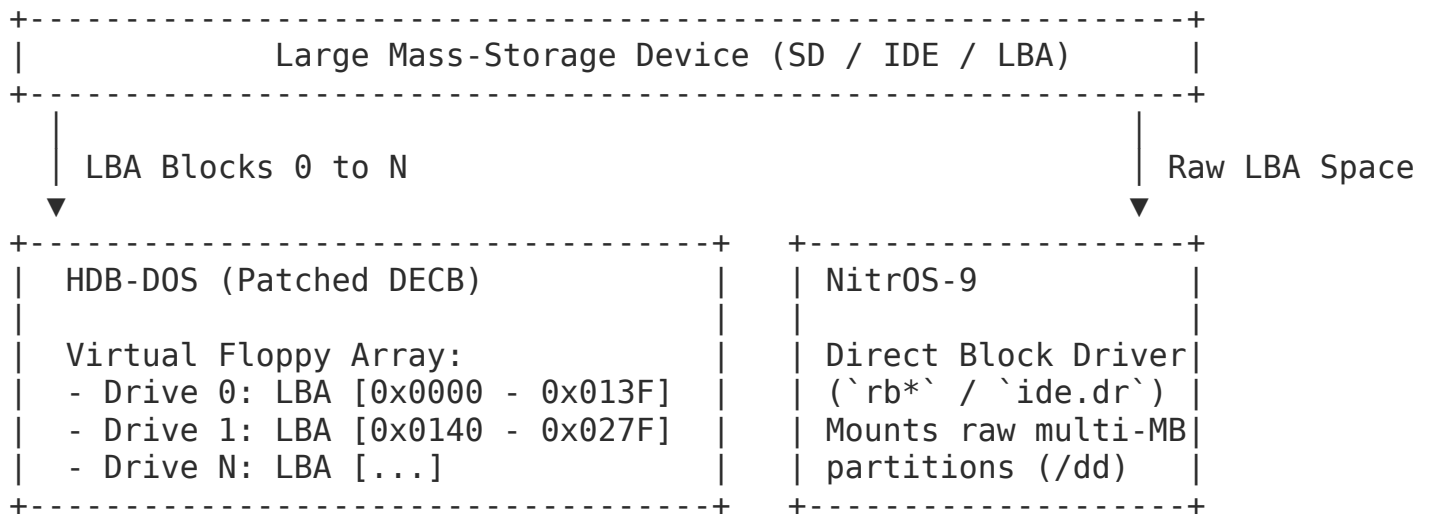
Why Unpatched DECB Cannot Use Large Drives

Stock DECB v1.1 is hardcoded around floppy disk geometry and register layouts:

1. **Addressing Limitations:** DECB addresses data using 1-byte Track (0–34) and 1-byte Sector (1–18) values. The directory structure and File Allocation Table (FAT) assume a fixed 68-granule disk (each granule being 5 sectors = 1,280 bytes). It cannot natively address linear LBA sectors or hard drive partitions.
2. **Direct Hardware Coupling:** DECB's low-level read/write primitives write directly to the WD17xx registers at \$FF48–\$FF4B. It has no abstraction layer to communicate with an IDE/ATA register bank or a SCSI host adapter.

How to Bridge the Two (The HDB-DOS Pattern)

To allow both Disk BASIC and NitrOS-9 to share a large drive mapped into the \$FF40–\$FF5F expansion space:



1. **Virtual Floppy Allocation for BASIC:** Instead of rewriting the entire BASIC FAT filesystem, patched ROMs like **HDB-DOS** hook DECB's low-level sector transfer vector (DSKCON at \$015E in RAM). When BASIC requests Track T , Sector S , Drive D , the patch calculates an LBA offset:

$$\text{LBA} = (\text{Virtual Drive Number} \times \text{Sectors Per Disk}) + (T \times 18) + (S - 1)$$

This treats a multi-gigabyte card as a container holding hundreds or thousands of virtual 156 KB floppy images without rewriting the BASIC high-level file commands.

2. **Native Raw Block Access for NitrOS-9:** NitrOS-9 does not care about floppy track/sector limits. It uses an installable block device driver (such as `rbsuper`, `rbco3`, or custom driver modules). The OS-9 driver talks directly to your custom I/O registers in \$FF40–\$FF5F, passing 24-bit or 32-bit linear LBA addresses and transferring raw 256-byte or 512-byte sectors directly into system data buffers.
3. **Coexistence via Partitioning:** By convention, a designated region of the drive (e.g., LBA 0 through some offset) is reserved for HDB-DOS virtual floppy disks, while the remainder is partitioned as an OS-9 filesystem partition mounted as root (`/dd`). This allows instant switching between DECB virtual disks and NitrOS-9 on the same storage medium.

User prompt: I think this basic DECB format is "double-density", right? `A standard .DSK file is just a flat array of 161,280 bytes (35 tracks  $\times$  18 sectors  $\times$  256 bytes)`. Don't double-sided drives exist? How are sectors and tracks numbered on double-sided drives? What BASIC's and OS's for Coco2 and Coco3 can handle them?

Response: Yes, that standard 161,280-byte format is **Double Density (MFM)**, but it is **Single-Sided** (often designated **SSDD** — Single-Sided Double Density).

- **Single Density (FM):** 10 sectors/track $\times$ 256 bytes = 2.5 KB/track (~87.5 KB on a 35-track disk). CoCo controllers used MFM by default, though the hardware could generate FM via the density bit in `$FF40`.
- **Double Density (MFM):** 18 sectors/track $\times$ 256 bytes = 4.5 KB/track (35×4.5 KB = 157.5 KB, or 161,280 bytes).

Double-sided drives (DSDD) and 40-track/80-track drives absolutely exist and were widely used in the CoCo community.

How Double-Sided Drives Handle Sectors and Tracks

Hardware and software handle sides using two distinct methodologies:

1. Hardware Level (Physical Addressing)

At the physical drive mechanism and controller level:

- **Track (Cylinder):** 0 to 34 (or 0 to 39 for 40-track drives; 0 to 79 for 80-track drives).
- **Head / Side:** Selected via **Bit 6 of `$FF40`** (0 = Side 0 / bottom head, 1 = Side 1 / top head).
- **Sectors:** Physically recorded on each track from **1 to 18**. Both sides have sectors numbered 1 through 18.

2. Software Level (Logical Addressing)

Because stock Color BASIC only exposes a single 1-byte Track parameter (0–255) and 1-byte Sector parameter (1–255) through its low-level sector driver (DSKCON), third-party DOSes had to decide how to represent side 2 to the file system. Two main schemes emerged:

- **Split-Drive / Twin-Drive Mapping (The Stock / Patch approach):** Instead of expanding disk capacity as a single monolithic drive, Side 1 is treated as an entirely separate virtual drive.
 - Physical Drive 0, Side 0 $\rightarrow$ Drive 0 (Tracks 0–34, Sectors 1–18)
 - Physical Drive 0, Side 1 $\rightarrow$ Drive 2 (Tracks 0–34, Sectors 1–18)
 - Physical Drive 1, Side 0 $\rightarrow$ Drive 1
 - Physical Drive 1, Side 1 $\rightarrow$ Drive 3 The user accesses the other side by simply changing drive letters (DIR 2).

- **Pseudo-Track Continuation (Linear Cylinder Scheme):** The DOS treats the entire disk as a single 70-track or 80-track volume under one drive number.
 - Logical Tracks 0–34 (or 0–39) = Physical Side 0
 - Logical Tracks 35–69 (or 40–79) = Physical Side 1 When the software calculates Logical Track ≥ 35 , the driver automatically asserts the side bit in \$FF40 (Bit 6) and steps the physical head to (Logical Track - 35). In flat .DSK files on an emulator, this is typically laid out linearly: all of Side 0 followed by all of Side 1, or interlaced cylinder-by-cylinder ($T_0S_0, T_0S_1, T_1S_0, T_1S_1 \dots$).

Which Operating Systems and BASICs Support Double-Sided Drives?

Stock Tandy DECB (v1.0 or v1.1) hardcodes disk operations to 35 tracks, 1 side, and 68 granules. To use double-sided or 40/80-track drives, you need patched DOSes or advanced operating systems:

On the BASIC Side (CoCo 2 and CoCo 3)

DOS / BASIC Variant	Compatibility	Mechanism & Features
RGB-DOS	CoCo 2 & 3	One of the most popular retro DOSes. Supports DSDD 40-track and 80-track drives (up to 720 KB 3.5" disks). Treats double-sided media as single large volumes with expanded FATs and dynamic directory allocation.
ADOS (Advanced DOS)	CoCo 1, 2 & 3	Highly configurable ROM replacement by Arthur Flexser. Maps the reverse side as an offset drive or extended tracks; supports 40-track double-sided drives seamlessly.
Super-DOS	CoCo 1 & 2	Popular early patch kit supporting 40-track and double-sided operations via side-switching commands.
SDC-DOS (CoCoSDC)	CoCo 2 & 3	Darren Atkinson's DOS firmware for the CoCoSDC; includes built-in commands to mount multi-track, multi-side images and toggle between split-drive and extended-track mappings.
HDB-DOS	CoCo 2 & 3	While primarily used to map hard-drive LBAs to virtual floppies, HDB-DOS variants support double-sided geometries when interacting with physical FDC hardware.

On the OS Side (Advanced Operating Systems)

- **OS-9 Level 1 (CoCo 1 & 2) and OS-9 / NitroOS-9 Level 2 (CoCo 3):** OS-9 was built from day one with a hardware-abstracted, device-independent I/O system. It does not care about fixed track or sector counts:
 - Floppy geometry is defined inside a **Device Descriptor** module (e.g., d0, dd).

- In the descriptor table, bytes configure:
 - **IT.CYL** (Number of cylinders: 35, 40, or 80)
 - **IT.SID** (Number of surfaces/heads: 1 or 2)
 - **IT.SCT** (Sectors per track: 18)
 - Interleave and step rates
 - The floppy driver (`rb1773` or `cc3disk`) reads these descriptor bytes and automatically handles stepping and flipping Bit 6 of `$FF40`. A standard 40-track DSDD disk gives OS-9 **360 KB** of contiguous storage, and an 80-track 3.5" DSDD disk yields **720 KB**.
 - **FLEX (Technical Systems Consultants)**: FLEX 9 was ported to the CoCo by Frank Hogg Laboratories and others. FLEX uses a two-byte Track/Sector pointer chain linked directly into each sector payload; its disk drivers natively handle 40-track and 80-track double-sided drives by configuring the low-level step/side lookup tables.
-

User prompt: In NitroS-9, how does Logical Sector Numbering (LSN) map onto track/side/sector numbering?

Response: In NitroS-9 (and standard Microware OS-9 Level 1/2), the file manager (RBF — Random Block File Manager) treats all block devices as flat, linear arrays of 256-byte blocks indexed by a 24-bit **Logical Sector Number (LSN)**, running from 0 up to $N - 1$.

The low-level disk driver (e.g., `rb1773`, `cc3disk`) translates this linear LSN into physical **Cylinder (Track)**, **Head (Side)**, and **Physical Sector** numbers on the fly using disk geometry parameters stored in the device's **Device Descriptor** (or read from the disk's Identification Sector at LSN 0).

1. The Descriptor Geometry Parameters

The driver reads these key fields out of the drive table / descriptor initialization block:

- **IT.SCT** (16-bit, though typically ≤ 255 for floppies): Number of sectors per track. For standard CoCo MFM disks, this is **18**.
- **IT.SID** (8-bit): Number of surfaces/heads. **1** for single-sided (SS), **2** for double-sided (DS).
- **IT.CYL** (16-bit): Total number of tracks/cylinders (e.g., 35, 40, or 80).

From these, the driver derives the number of sectors per cylinder:

$$\text{Sectors Per Cylinder} = \text{IT.SCT} \times \text{IT.SID}$$

2. The Standard Mapping Algorithm (Cylinder-Interleaved)

NitroS-9 formats and traverses multi-sided disks by **cylinder first**, not side first. The drive reads all sectors on Track T , Side 0, then flips the electronic head switch to Side 1 on that *same* cylinder

before stepping the physical stepper motor to Track $T + 1$. This minimizes slow mechanical head-stepping latency.

Given a 24-bit LSN, the driver uses integer division and modulo arithmetic:

Step 1: Compute Cylinder (Physical Track)

$$\text{Cylinder} = \left\lfloor \frac{\text{LSN}}{\text{IT.SCT} \times \text{IT.SID}} \right\rfloor$$

Step 2: Compute Remainder within the Cylinder

$$\text{Cylinder_Offset} = \text{LSN} \pmod{\text{IT.SCT} \times \text{IT.SID}}$$

Step 3: Compute Head (Side)

$$\text{Head} = \left\lfloor \frac{\text{Cylinder_Offset}}{\text{IT.SCT}} \right\rfloor$$

(For single-sided disks where $\text{IT.SID} = 1$, this always evaluates to 0).

Step 4: Compute Physical Sector

Floppy controllers (like the WD1773/WD1793) use **1-based sector numbering** (1 . . . 18), whereas LSN is **0-based** (0 . . . Max).

$$\text{Sector} = (\text{Cylinder_Offset} \pmod{\text{IT.SCT}}) + 1$$

3. Worked Concrete Example: 40-Track DSDD Disk

Parameters:

- $\text{IT.SCT} = 18$
- $\text{IT.SID} = 2$
- $\text{Sectors Per Cylinder} = 18 \times 2 = 36$

LSN (0-based)	Cylinder (Track)	Head (Side)	Physical Sector (1-based)	Notes
0	Track 0	Side 0	Sector 1	Disk Identification Sector (DD.T0T, etc.)
1	Track 0	Side 0	Sector 2	Allocation Map / Root Directory entry

LSN (0-based)	Cylinder (Track)	Head (Side)	Physical Sector (1-based)	Notes
...	...	...	...	
17	Track 0	Side 0	Sector 18	Last sector of Cylinder 0, Side 0
18	Track 0	Side 1	Sector 1	Head flips to Side 1; no motor step!
19	Track 0	Side 1	Sector 2	
...	...	...	...	
35	Track 0	Side 1	Sector 18	Last sector of Cylinder 0, Side 1
36	Track 1	Side 0	Sector 1	Physical head steps inward to Track 1
37	Track 1	Side 0	Sector 2	

4. Hardware Implementation in Driver Code

In the 6809 driver routines:

1. **Division:** The 6809 lacks a 32-bit or 24-bit division instruction (it only has 16-bit by 8-bit IDIV). The driver performs a quick loop or shifts if `IT.SCT` is a power of 2, or standard multi-byte subtract-and-shift division to isolate the track, side, and sector.
2. **Writing Hardware Registers:**
 - **Cylinder:** Loaded into the 6809 accumulator and compared against the current track register. If different, a Seek or Step command is sent to the WD controller register at `$FF48`.
 - **Head/Side:** Bit 6 of the `$FF40` control latch is updated (0 for side 0, 1 for side 1).
 - **Sector:** Written to the WD Sector Register at `$FF4A` (with the `+1` bias added).

5. Sector Interleave (`IT.ITL`)

NitrOS-9 formatters (`format`) also account for **interleave** (usually 3:1 or 4:1 for 0.89 MHz systems, or 1:1 on accelerated/RAM/emulated setups).

On physical media, interleave is typically handled during formatting by skewing the physical sector numbers written in the sector header IDs on the track (e.g., recording sector 1, then sector 7, then sector 13 in sequence). Because the physical track already contains skewed sector IDs, the driver's

simple mathematical LSN translation ($1 \dots 18$) remains unchanged during runtime reads and writes.
